

Assignment 3. ETL with BASH

Chioma Onyekpere

Predictive Analytics Diploma, PACE, University of Winnipeg

Data Acquisition & Management for Data Science

Muhammad Shahin

February 9, 2025

lab3

February 15, 2025

1 ETL with Bash

1.1 Objectives

The objective of this assignment is to implement an **ETL (Extract, Transform, Load) process using Bash scripting**. This exercise focuses on automating data extraction, transformation, and loading into a relational database using shell scripting techniques. The key learning goals are:

1. **Extracting Data:** Ingest and extract data from a remote source using Bash commands like `curl` or `wget`.
2. **Transforming Data:** Process and manipulate the extracted data using shell scripting tools (e.g., `cut`, `awk`, `sed`).
3. **Loading Data:** Import the transformed data into a **PostgreSQL database** using a Python script that creates a database table and loads the dataset.
4. **Scheduling the ETL Job:** Automate the ETL pipeline to run daily at a specific time using `cron` (job scheduler).

This assignment provides hands-on experience in **shell scripting, database interaction, data transformation, and job scheduling**, essential skills for data engineering roles.

1.2 Approach and Methodology

The ETL pipeline is implemented using a structured approach that follows these key steps:

1. **Extract Phase:**
 - Use Bash commands (`curl` or `wget`) to download the dataset from a remote URL.
 - Implement error handling to ensure the dataset is successfully retrieved before proceeding.
2. **Transform Phase:**
 - Process the extracted dataset using tools like `cut`, `awk`, and `sed` to select specific columns and clean the data.
 - Ensure consistency and format the data appropriately for database insertion.
3. **Load Phase:**
 - Execute a Python script to connect to a PostgreSQL database and insert the transformed data.
 - The script will create the necessary table (if it does not exist) and load the cleaned dataset.
4. **Automation with Cron:**
 - Schedule the Bash script to run at a fixed time daily using `crontab`.
 - Ensure logs are generated for tracking execution success or failures.

This methodology ensures that the ETL process runs efficiently and can be easily maintained or modified for different datasets.

1.3 Expected Outcomes

Upon completing this assignment, the following outcomes are anticipated:

- A fully functional **ETL pipeline** that automates data extraction, transformation, and loading into a PostgreSQL database.
- A **Bash script** that efficiently processes and cleans the data.
- A **Python script** that seamlessly interacts with the database to insert transformed data.
- A scheduled job using **cron**, ensuring the ETL process runs daily without manual intervention.
- Improved understanding of **shell scripting, automation, and data processing techniques**.
- Experience handling **real-world datasets and integrating Bash with database systems**.

This assignment will help build **practical data engineering skills** applicable to real-world ETL workflows.

I will be redoing this assignment on my mac. Kept running into issues on my vm ubuntu earlier today

1.4 Task 1: Creating a Bash Script File and Setting Permissions

1.4.1 Step 1: Create the Bash Script File

To begin the ETL process, we create a Bash script file named `ETL_script.sh`:

```
touch ETL_script.sh
```

This command creates an empty file that will store the Bash commands for the ETL pipeline.

1.4.2 Step 2: Check File Permissions

To verify the default permissions of the file, use the `ls -l` command:

```
ls -l ETL_script.sh
```

Output:

```
-rw-r--r--  1 chiomaonyekpere  staff   0 Feb 15 19:04 ETL_script.sh
```

Explanation: - **rw** → Owner has **read and write** permissions. - **r** → Group members have **read-only** permission. - **r--** → Others have **read-only** permission. - The **execute (x)** permission is **missing**, meaning the script cannot be executed yet.

1.4.3 Step 3: Grant Execute Permission

To allow the script to be executed, update its permissions using `chmod`:

```
chmod +x ETL_script.sh
```

This command adds **execute (x)** permission for the owner, making the script executable.

1.4.4 Step 4: Verify Updated Permissions

Re-run the `ls -l` command to confirm the change:

```
ls -l ETL_script.sh
```

Expected output:

```
-rwxr-xr-x  1 chiomaonyekpere  staff  0 Feb 15 19:04 ETL_script.sh
```

Explanation: - rwx → Owner can **read, write, and execute**. - xr → Group members can **read and execute***. - x → Others can **execute**.

```
[41]: from IPython.display import display, Image

display(Image(filename="screenshots/create_file.png"))
```

```
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % pwd
/Users/chiomaonyekpere/Documents/PACE/data-science-python-practice/term_two_winter/data_acquisition_and_management_for_data_science/lab3
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % ls
lab3.ipynb  screenshots
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % touch ETL_script.sh
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % ls -l ETL_script.sh
-rw-r--r--  1 chiomaonyekpere  staff  0 Feb 15 19:04 ETL_script.sh
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % chmod +x ETL_script.sh
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % ls -l ETL_script.sh
-rwxr-xr-x  1 chiomaonyekpere  staff  0 Feb 15 19:04 ETL_script.sh
• (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % cat ETL_script.sh
○ (base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 %
```

1.5 Task 2: Opening the Script in a Text Editor

1.5.1 Step 1: Open the Script in VS Code

I will be working from VS Code this time instead of `nano` for convenience:

code `ETL_script.sh`

- This will open the file inside the **VS Code editor**.

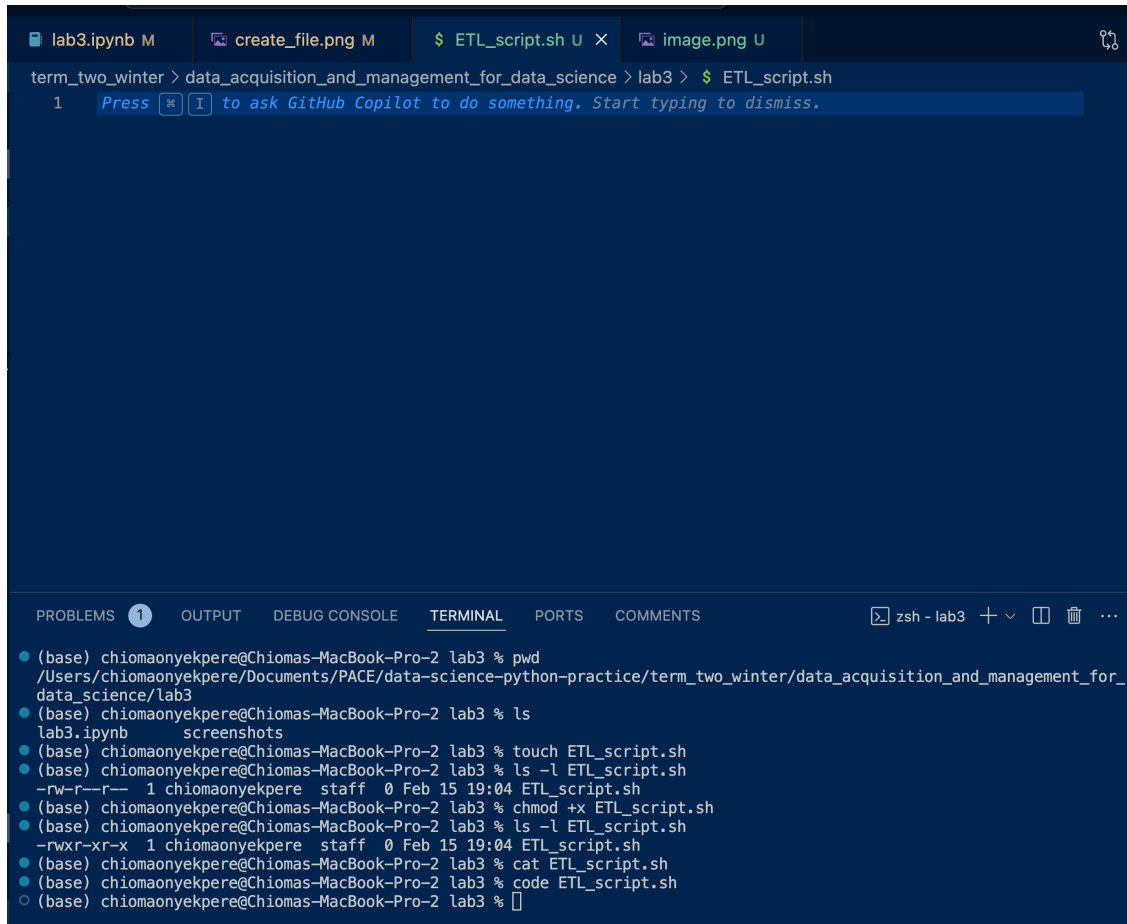
Alternative Editors: - **Vim:** `vim ETL_script.sh` - **nano:** `nano ETL_script.sh`

1.5.2 Step 2: Confirm the File is Opened

Once the command is executed, the terminal should display an **empty editor** where the script can be written.

```
[42]: # open file in nano command nano ETL_script.sh

display(Image(filename="screenshots/code_open_command.png"))
```



```
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > $ ETL_script.sh
1 Press [⌘][I] to ask GitHub Copilot to do something. Start typing to dismiss.

(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % pwd
/Users/chiomaonyekpere/Documents/PACE/data-science-python-practice/term_two_winter/data_acquisition_and_management_for_data_science/lab3
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ls
lab3.ipynb  screenshots
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % touch ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ls -l ETL_script.sh
-rw-r--r-- 1 chiomaonyekpere staff 0 Feb 15 19:04 ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % chmod +x ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ls -l ETL_script.sh
-rwxr-xr-x 1 chiomaonyekpere staff 0 Feb 15 19:04 ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % cat ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % code ETL_script.sh
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 %
```

1.6 Task 3: Initializing the Bash Script and Defining Variables

1.6.1 Step 1: Add the Shebang (#!)

The **shebang** (**#!**) specifies the interpreter that will execute the script. Add the following line at the top of the file:

```
#!/bin/bash
```

This ensures that the script runs using the **Bash shell**.

1.6.2 Step 2: Define Variables

To maintain clarity and efficiency, define key variables at the beginning of the script:

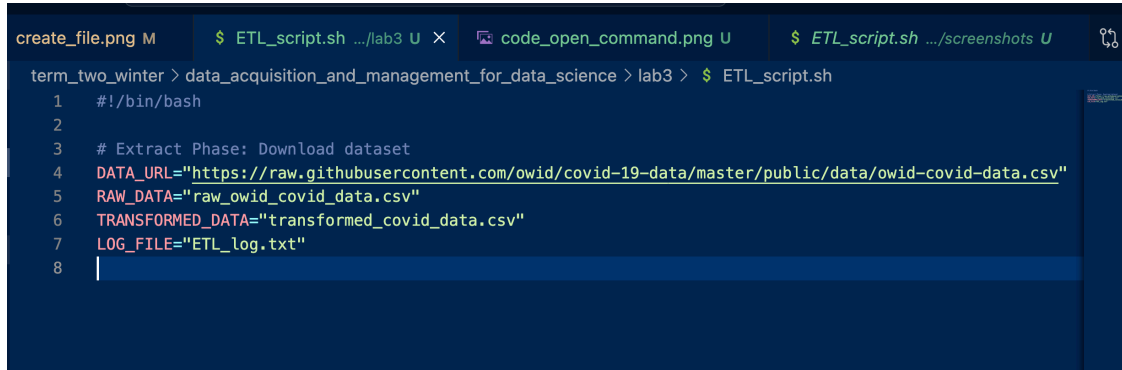
```
# Define variables for dataset URL, filenames, and log file
DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-d
RAW_DATA="raw_owid_covid_data.csv"
TRANSFORMED_DATA="transformed_covid_data.csv"
LOG_FILE="ETL_log.txt"
```

- **DATA_URL** → Stores the URL of the dataset to be downloaded.

- RAW_DATA → Specifies the name of the file to store extracted data.
- TRANSFORMED_DATA → Stores the filename of the cleaned, processed data.
- LOG_FILE → Keeps track of ETL execution logs.

[43]: *# editing file in nano*

```
display(Image(filename="screenshots/define_vars.png"))
```



```

create_file.png M    $ ETL_script.sh .../lab3 U X    code_open_command.png U    $ ETL_script.sh .../screenshots U
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > $ ETL_script.sh
1  #!/bin/bash
2
3  # Extract Phase: Download dataset
4  DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv"
5  RAW_DATA="raw_owid_covid_data.csv"
6  TRANSFORMED_DATA="transformed_covid_data.csv"
7  LOG_FILE="ETL_log.txt"
8

```

1.7 4. Task 4: Extracting Data - Downloading the File

1.7.1 Step 1: Define the Download Process

The Extract phase involves downloading the latest version of the dataset each time the script is executed. This ensures that the data used in the pipeline is always up to date. The dataset is downloaded from the specified remote location.

1.7.2 Step 2: Implement the Download Script

The following script performs the extraction step:

```

# Extract Phase: Download dataset
DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv"
RAW_DATA="raw_owid_covid_data.csv"
TRANSFORMED_DATA="transformed_covid_data.csv"
LOG_FILE="ETL_log.txt"

# Log the start of the extraction process
echo "$(date): Starting data extraction..." >> $LOG_FILE

# Download the dataset (overwrite existing file to get the latest data)
curl -o $RAW_DATA $DATA_URL

# Verify if the file was downloaded successfully
if [[ -f "$RAW_DATA" ]]; then
    ROW_COUNT=$((wc -l < "$RAW_DATA" - 1))
    COL_COUNT=$(head -1 "$RAW_DATA" | awk -F',' '{print NF}')

```

```

echo "$date): Data extraction successful. Rows extracted: $ROW_COUNT, Columns extracted: $COL
else
    echo "$date): Data extraction failed" >> $LOG_FILE
    exit 1
fi

```

1.7.3 Explanation of the Script

1. Defines Variables:

- DATA_URL stores the dataset's URL.
- RAW_DATA defines the filename for storing the extracted data.
- LOG_FILE keeps track of execution steps.

2. Logs the Start of Extraction: The script logs when the download starts.

3. Downloads the File: The curl command fetches the dataset and overwrites any previous file.

4. Verifies the Download:

- If the file exists after the download, it logs success.
- If the file is missing, it logs the failure and exits the script to prevent further execution errors.

1.7.4 Step 3: Run the ETL Script

```
./ETL_script.sh
```

1.7.5 What Happens?

1. The script **download** the COVID-19 dataset.
2. It **logs** the extraction process into ETL_log.txt.
3. I verify if the file exists by running:

```
ls -l raw_owid_covid_data.csv
```

1.7.6 Output

```
-rw-r--r--  1 chiomaonyekpere  staff  98391483 Feb 15 19:36 raw_owid_covid_data.csv
```

And the log file:

```
Mon Feb 10 02:49:40 PM UTC 2025: Starting data extraction...
```

```
Mon Feb 10 02:49:44 PM UTC 2025: Data extraction successful
```

```
[44]: #download script
```

```
display(Image(filename="screenshots/download_raw_data_script.png"))
```

```

lab3.ipynb M create_file.png M $ ETL_script.sh .../lab3 U X download_raw_data_script.png M ne
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > $ ETL_script.sh
1  #!/bin/bash
2
3  # Extract Phase: Download dataset
4  DATA_URL="https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv"
5  RAW_DATA="raw_owid_covid_data.csv"
6  TRANSFORMED_DATA="transformed_covid_data.csv"
7  LOG_FILE="ETL_log.txt"
8
9  # Log the start of the extraction process
10 echo "$(date): Starting data extraction..." >> $LOG_FILE
11
12 # Download the dataset (overwrite existing file to get the latest data)
13 curl -o $RAW_DATA $DATA_URL
14
15 # Verify if the file was downloaded successfully
16 if [[ -f "$RAW_DATA" ]]; then
17     ROW_COUNT=$((wc -l < "$RAW_DATA" - 1))
18     COL_COUNT=$(head -1 "$RAW_DATA" | awk -F',' '{print NF}')
19     echo "$(date): Data extraction successful. Rows extracted: $ROW_COUNT, Columns extracted: $COL_COUNT" >>
20 else
21     echo "$(date): Data extraction failed" >> $LOG_FILE
22     exit 1
23 fi

```

```

(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ./ETL_script.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 93.8M  100 93.8M    0     0  6883k      0  0:00:13  0:00:13 --:--:-- 9567k
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % cat ETL_log.txt
Sat Feb 15 19:58:54 CST 2025: Starting data extraction...
Sat Feb 15 19:59:08 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ls -l raw_owid_covid_data.csv
-rw-r--r-- 1 chiomaonyekpere staff 98391483 Feb 15 19:59 raw_owid_covid_data.csv
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 %

```

```

[53]: import pandas as pd

# Define the file path
raw_owid_covid_data = "raw_owid_covid_data.csv"

# Load the CSV file
df = pd.read_csv(raw_owid_covid_data)

# Display a sample of the data
print("Sample Data:")
df.head()

```

Sample Data:

```

[53]:   iso_code  continent  location  date  total_cases  new_cases  \
0      AFG         Asia  Afghanistan  2020-01-05         0.0         0.0
1      AFG         Asia  Afghanistan  2020-01-06         0.0         0.0
2      AFG         Asia  Afghanistan  2020-01-07         0.0         0.0
3      AFG         Asia  Afghanistan  2020-01-08         0.0         0.0

```


4	AFG	Asia	Afghanistan	2020-01-09	0.0	0.0
---	-----	------	-------------	------------	-----	-----

	new_cases_smoothed	total_deaths	new_deaths	new_deaths_smoothed	...	\
0	NaN	0.0	0.0	NaN	...	
1	NaN	0.0	0.0	NaN	...	
2	NaN	0.0	0.0	NaN	...	
3	NaN	0.0	0.0	NaN	...	
4	NaN	0.0	0.0	NaN	...	

	male_smokers	handwashing_facilities	hospital_beds_per_thousand	\
0	NaN	37.75	0.5	
1	NaN	37.75	0.5	
2	NaN	37.75	0.5	
3	NaN	37.75	0.5	
4	NaN	37.75	0.5	

	life_expectancy	human_development_index	population	\
0	64.83	0.51	41128772	
1	64.83	0.51	41128772	
2	64.83	0.51	41128772	
3	64.83	0.51	41128772	
4	64.83	0.51	41128772	

	excess_mortality_cumulative_absolute	excess_mortality_cumulative	\
0	NaN	NaN	
1	NaN	NaN	
2	NaN	NaN	
3	NaN	NaN	
4	NaN	NaN	

	excess_mortality	excess_mortality_cumulative_per_million
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

[5 rows x 67 columns]

```
[54]: # Show basic dataset information
num_rows, num_cols = df.shape
print(f"\nTotal Rows: {num_rows}")
print(f"Total Columns: {num_cols}")
```

Total Rows: 429435
Total Columns: 67

```
[55]: # Display column names
print("\nColumn Names:", list(df.columns))
```

```
Column Names: ['iso_code', 'continent', 'location', 'date', 'total_cases',
'new_cases', 'new_cases_smoothed', 'total_deaths', 'new_deaths',
'new_deaths_smoothed', 'total_cases_per_million', 'new_cases_per_million',
'new_cases_smoothed_per_million', 'total_deaths_per_million',
'new_deaths_per_million', 'new_deaths_smoothed_per_million',
'reproduction_rate', 'icu_patients', 'icu_patients_per_million',
'hosp_patients', 'hosp_patients_per_million', 'weekly_icu_admissions',
'weekly_icu_admissions_per_million', 'weekly_hosp_admissions',
'weekly_hosp_admissions_per_million', 'total_tests', 'new_tests',
'total_tests_per_thousand', 'new_tests_per_thousand', 'new_tests_smoothed',
'new_tests_smoothed_per_thousand', 'positive_rate', 'tests_per_case',
'tests_units', 'total_vaccinations', 'people_vaccinated',
'people_fully_vaccinated', 'total_boosters', 'new_vaccinations',
'new_vaccinations_smoothed', 'total_vaccinations_per_hundred',
'people_vaccinated_per_hundred', 'people_fully_vaccinated_per_hundred',
'total_boosters_per_hundred', 'new_vaccinations_smoothed_per_million',
'new_people_vaccinated_smoothed', 'new_people_vaccinated_smoothed_per_hundred',
'stringency_index', 'population_density', 'median_age', 'aged_65_older',
'aged_70_older', 'gdp_per_capita', 'extreme_poverty', 'cardiovasc_death_rate',
'diabetes_prevalence', 'female_smokers', 'male_smokers',
'handwashing_facilities', 'hospital_beds_per_thousand', 'life_expectancy',
'human_development_index', 'population', 'excess_mortality_cumulative_absolute',
'excess_mortality_cumulative', 'excess_mortality',
'excess_mortality_cumulative_per_million']
```

```
[56]: # Show data types of each column
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
iso_code          object
continent         object
location          object
date              object
total_cases       float64
...
population        int64
excess_mortality_cumulative_absolute float64
excess_mortality_cumulative         float64
excess_mortality                    float64
excess_mortality_cumulative_per_million float64
Length: 67, dtype: object
```

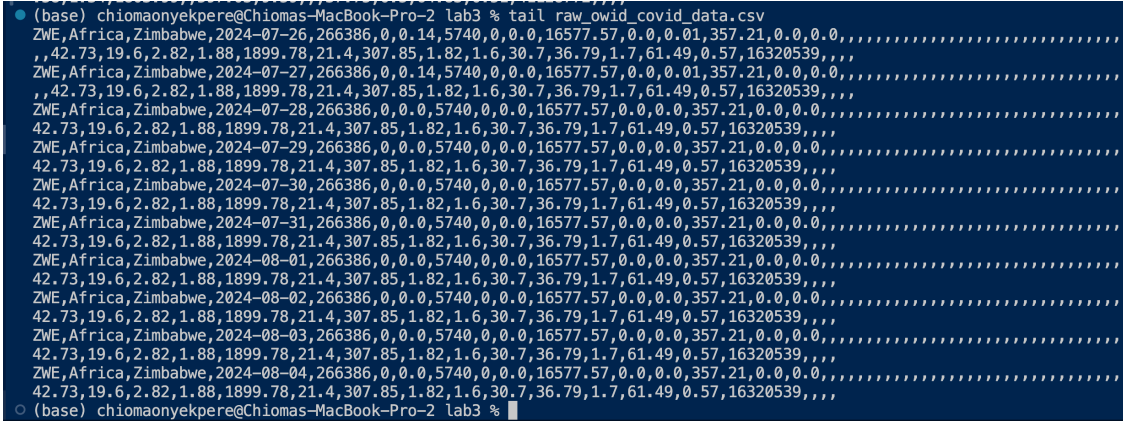
```
[57]: # Check for missing values
missing_values = df.isnull().sum()
print("\nMissing Values:")
print(missing_values[missing_values > 0])
```

```
Missing Values:
continent                26525
total_cases              17631
new_cases                19276
new_cases_smoothed      20506
total_deaths             17631
...
human_development_index 110308
excess_mortality_cumulative_absolute 416024
excess_mortality_cumulative 416024
excess_mortality         416024
excess_mortality_cumulative_per_million 416024
Length: 63, dtype: int64
```

```
[58]: # view first 10 lines of csv
display(Image(filename="screenshots/head_row_data.png"))
```

[illegible]

```
[59]: # view last 10 lines of csv
display(Image(filename="screenshots/tail_raw_data.png"))
```



```
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % tail raw_owid_covid_data.csv
ZWE,Africa,Zimbabwe,2024-07-26,266386,0.0,14,5740,0.0,0.0,16577.57,0.0,0.01,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
,42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-07-27,266386,0.0,14,5740,0.0,0.0,16577.57,0.0,0.01,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
,42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-07-28,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-07-29,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-07-30,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-07-31,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-08-01,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-08-02,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-08-03,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
ZWE,Africa,Zimbabwe,2024-08-04,266386,0.0,0,5740,0.0,0.0,16577.57,0.0,0.0,357.21,0.0,0.0,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
42.73,19.6,2.82,1.88,1899.78,21.4,307.85,1.82,1.6,30.7,36.79,1.7,61.49,0.57,16320539,,,,,
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 %
```

1.8 Task 5: Transforming Data

1.8.1 Step 1: Extract Specific Columns

The transformation step involves selecting specific columns (1, 2, 4, and 5) from the dataset. This can be achieved using the `cut` command:

```
cut -d',' -f1,2,4,5 raw_owid-covid-data.csv > transformed_data.csv
```

- The `-d','` option specifies that the **delimiter** is a comma.
- The `-f1,2,4,5` option selects only **columns 1, 2, 4, and 5**.
- The output is saved to `transformed_data.csv`.

1.8.2 Step 2: Save the Transformation Process in the Script

Modify `ETL_script.sh` and add the transformation step inside it:

```
# Transform Phase: Select specific columns
TRANSFORMED_DATA="transformed_data.csv"
echo "$(date): Starting data transformation..." >> $LOG_FILE
cut -d',' -f1,2,4,5 $RAW_DATA > $TRANSFORMED_DATA

# Verify if transformation was successful
if [[ -f "$TRANSFORMED_DATA" ]]; then
    echo "$(date): Data transformation successful" >> $LOG_FILE
else
    echo "$(date): Data transformation failed" >> $LOG_FILE
fi
```

1.9 5. Task 5: Transforming Data

1.9.1 Step 1: Define the Transformation Process

The transformation phase focuses on selecting specific columns from the extracted dataset to retain only relevant information. This helps streamline data processing and ensures that only necessary data is stored in the database.

1.9.2 Step 2: Implement the Transformation Script

The transformation process uses the `cut` command to extract columns **1, 2, 4, and 5**, which correspond to `iso_code`, `continent`, `date`, and `total_cases`.

```
# Transform Phase: Select specific columns
TRANSFORMED_DATA="transformed_covid_data.csv"
echo "$(date): Starting data transformation..." >> $LOG_FILE

# Select only required columns
cut -d',' -f1,2,4,5 $RAW_DATA > $TRANSFORMED_DATA

# Count the number of transformed rows (excluding header)
ROW_COUNT=$((wc -l < "$TRANSFORMED_DATA" - 1))

# Verify if transformation was successful
if [[ -f "$TRANSFORMED_DATA" ]]; then
    echo "$(date): Data transformation successful. Rows transformed: $ROW_COUNT" >> $LOG_FILE
else
    echo "$(date): Data transformation failed" >> $LOG_FILE
    exit 1
fi
```

1.9.3 Step 3: Explanation of the Script

1. **Defines Variables:**
 - `TRANSFORMED_DATA` stores the name of the transformed output file.
2. **Logs the Start of Transformation:** The script logs when the transformation process begins.
3. **Extracts Relevant Columns:** Uses `cut` to select specific columns from the extracted dataset.
4. **Counts the Number of Transformed Rows:** Uses `wc -l` to count the number of records (excluding headers) and logs this information.
5. **Verifies the Success of Transformation:**
 - If the transformed file exists, it logs success and the row count.
 - If the file is missing, it logs failure and exits the script to prevent further errors.

This ensures that the transformed dataset is properly formatted and ready for loading into PostgreSQL.

```
[60]: # show transformed csv
```

```
display(Image(filename="screenshots/transformed_csv.png"))
```

```

$ ETL_script.sh .../lab3 U X
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > $ ETL_script.sh
24
25 # Transform Phase: Select specific columns
26 echo "$(date): Starting data transformation..." >> $LOG_FILE
27
28 # Select only required columns
29 cut -d',' -f1,2,4,5 $RAW_DATA > $TRANSFORMED_DATA
30
31 # Count the number of transformed rows (excluding header)
32 ROW_COUNT=$((wc -l < "$TRANSFORMED_DATA" - 1))
33
34 # Verify if transformation was successful
35 if [[ -f "$TRANSFORMED_DATA" ]]; then
36     COL_COUNT=$(head -1 "$TRANSFORMED_DATA" | awk -F',' '{print NF}')
37     echo "$(date): Data transformation successful. Rows transformed: $ROW_COUNT, Columns retained: $COL_COUNT"
38 else
39     echo "$(date): Data transformation failed" >> $LOG_FILE

```

```

(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ./ETL_script.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total   Spent    Left     Speed
100 93.8M  100 93.8M    0     0  6883k      0  0:00:13  0:00:13 --:--:-- 9567k
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % cat ETL_log.txt
Sat Feb 15 19:58:54 CST 2025: Starting data extraction...
Sat Feb 15 19:59:08 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ls -l raw_owid_covid_data.csv
-rw-r--r-- 1 chiomaonyekpere staff 98391483 Feb 15 19:59 raw_owid_covid_data.csv
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % ./ETL_script.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total   Spent    Left     Speed
100 93.8M  100 93.8M    0     0  19.8M      0  0:00:04  0:00:04 --:--:-- 20.6M
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % cat ETL_log.txt
Sat Feb 15 19:58:54 CST 2025: Starting data extraction...
Sat Feb 15 19:59:08 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
Sat Feb 15 20:02:03 CST 2025: Starting data extraction...
Sat Feb 15 20:02:07 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
Sat Feb 15 20:02:08 CST 2025: Starting data transformation...
Sat Feb 15 20:02:09 CST 2025: Data transformation successful. Rows transformed: 429435, Columns retained: 4
(base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 %

```

```
[64]: # Define the file path
transformed_covid_data = "transformed_covid_data.csv"

# Load the CSV file
transformed_df = pd.read_csv(transformed_covid_data)

# Display a sample of the data
print("Transformed Data:")
transformed_df.head()
```

Transformed Data:

```
[64]:
```

	iso_code	continent	date	total_cases
0	AFG	Asia	2020-01-05	0.0
1	AFG	Asia	2020-01-06	0.0
2	AFG	Asia	2020-01-07	0.0
3	AFG	Asia	2020-01-08	0.0

4 AFG Asia 2020-01-09 0.0

```
[65]: # Show basic dataset information
num_rows, num_cols = transformed_df.shape
print(f"\nTotal Rows: {num_rows}")
print(f"Total Columns: {num_cols}")
```

Total Rows: 429435
Total Columns: 4

```
[66]: # Display column names
print("\nColumn Names:", list(transformed_df.columns))
```

Column Names: ['iso_code', 'continent', 'date', 'total_cases']

```
[68]: # Show data types of each column
print("\nData Types:")
print(transformed_df.dtypes)
```

Data Types:

iso_code	object
continent	object
date	object
total_cases	float64

dtype: object

```
[69]: # Check for missing values
missing_values = transformed_df.isnull().sum()
print("\nMissing Values:")
print(missing_values[missing_values > 0])
```

Missing Values:

continent	26525
total_cases	17631

dtype: int64

1.10 Task 6: Loading Data into PostgreSQL

1.10.1 Step 1: Create a Python Script for Data Loading

To load the transformed data into a PostgreSQL database, we use a Python script that: 1. Connects to a PostgreSQL instance. 2. Creates a database (`covid_db`) and a table (`COVID`) if they do not exist. 3. Loads the transformed data using the `COPY` command.

Created a new Python script named `load_to_postgres.py` and add the following code:

```

import psycopg2
import os
import sys

# Define database connection parameters
db_name = os.getenv('DB_NAME', 'covid_db')
db_user = os.getenv('DB_USER', 'username')
db_password = os.getenv('DB_PASSWORD', 'password')
db_host = os.getenv('DB_HOST', 'localhost')
db_port = os.getenv('DB_PORT', '5432')

# Define input file
transformed_data = "transformed_data.csv"

conn = None
cursor = None

try:
    # Connect to PostgreSQL default database to check/create covid_db
    temp_conn = psycopg2.connect(
        dbname="postgres", user=db_user, password=db_password, host=db_host, port=db_port
    )
    temp_conn.autocommit = True
    temp_cursor = temp_conn.cursor()

    # Create the database if it doesn't exist
    temp_cursor.execute(f"SELECT 1 FROM pg_database WHERE datname = '{db_name}'")
    if not temp_cursor.fetchone():
        temp_cursor.execute(f"CREATE DATABASE {db_name}")

    temp_cursor.close()
    temp_conn.close()

    # Now, connect to the actual database
    conn = psycopg2.connect(
        dbname=db_name, user=db_user, password=db_password, host=db_host, port=db_port
    )
    cursor = conn.cursor()

    # Create table if it doesn't exist
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS COVID (
            iso_code VARCHAR(10),
            continent VARCHAR(50),
            date DATE,
            total_cases BIGINT
        );
    """)

```



```

    # Load data into the table
    with open(transformed_data, 'r') as file:
        cursor.copy_expert("COPY COVID FROM STDIN WITH CSV HEADER", file)

    conn.commit()
    print("Data successfully loaded into PostgreSQL.")

except psycopg2.OperationalError as e:
    print("Database connection error:", e)
    sys.exit(1)
except Exception as e:
    print("Error loading data:", e)
    sys.exit(1)
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()

```

1.10.2 Step 2: Modify ETL Script to Call Python Loader

Update ETL_script.sh to include the database loading step:

```

# Load Phase: Load transformed data into PostgreSQL
echo "$(date): Starting data load into PostgreSQL..." >> $LOG_FILE
python3 load_to_postgres.py

if [[ $? -eq 0 ]]; then
    echo "$(date): Data successfully loaded into PostgreSQL" >> $LOG_FILE
else
    echo "$(date): Data load failed" >> $LOG_FILE
fi

```

1.11 6. Task 6: Loading Data into PostgreSQL

1.11.1 Step 1: Create a Python Script for Data Loading

To load the transformed data into a PostgreSQL database, we use a Python script that: 1. Connects to a PostgreSQL instance. 2. Creates a database (covid_db) and a table (COVID) if they do not exist. 3. Loads the transformed data using the COPY command.

Create a new Python script named load_to_postgres.py and add the following code:

```

import psycopg2
import os
import sys
from datetime import datetime

# Define database connection parameters

```

```

db_name = os.getenv('DB_NAME', 'covid_db')
db_user = os.getenv('PSQL_USER', 'postgres')
db_password = os.getenv('DB_PASSWORD', 'postgres')
db_host = os.getenv('DB_HOST', 'localhost')
db_port = os.getenv('DB_PORT', '5432')

# Define input file
transformed_data = "transformed_covid_data.csv"

conn = None
cursor = None

try:
    # Connect to PostgreSQL default database to check/create covid_db
    temp_conn = psycopg2.connect(
        dbname="postgres", user=db_user, password=db_password, host=db_host, port=db_port
    )
    temp_conn.autocommit = True
    temp_cursor = temp_conn.cursor()

    # Create the database if it doesn't exist
    temp_cursor.execute(f"SELECT 1 FROM pg_database WHERE datname = '{db_name}'")
    if not temp_cursor.fetchone():
        temp_cursor.execute(f"CREATE DATABASE {db_name}")

    temp_cursor.close()
    temp_conn.close()

    # Now, connect to the actual database
    conn = psycopg2.connect(
        dbname=db_name, user=db_user, password=db_password, host=db_host, port=db_port
    )
    cursor = conn.cursor()

    # Create table if it doesn't exist
    cursor.execute("""
        SET search_path TO public;
        CREATE TABLE IF NOT EXISTS COVID (
            iso_code VARCHAR(10),
            continent VARCHAR(50),
            date DATE,
            total_cases BIGINT
        );
    """)

    # Load data into the table
    cursor.execute("DELETE FROM COVID;")
    with open(transformed_data, 'r') as file:

```

```

        cursor.copy_expert("COPY COVID FROM STDIN WITH CSV HEADER", file)

# Check row count after insertion
        cursor.execute("SELECT COUNT(*) FROM COVID;")
        after_insert = cursor.fetchone()[0]
        rows_inserted = after_insert

        conn.commit()
        print(f"Data successfully loaded. {rows_inserted} rows inserted into {db_name}. Database n")

# Log the inserted row count

        with open("ETL_log.txt", "a") as log_file:
            log_file.write(f"{datetime.now()}: Data successfully loaded. {rows_inserted} rows inser")

        conn.commit()
        print("Data successfully loaded into PostgreSQL.")

    except psycopg2.OperationalError as e:
        print("Database connection error:", e)
        sys.exit(1)
    except Exception as e:
        print("Error loading data:", e)
        sys.exit(1)
    finally:
        if cursor:
            cursor.close()
        if conn:
            conn.close()

```

1.11.2 Step 2: Modify ETL Script to Call Python Loader

Update ETL_script.sh to include the database loading step:

```

# Load Phase: Load transformed data into PostgreSQL
echo "$(date): Starting data load into PostgreSQL..." >> $LOG_FILE
python3 load_to_postgres.py

if [[ $? -eq 0 ]]; then
    echo "$(date): Data successfully loaded into PostgreSQL" >> $LOG_FILE
else
    echo "$(date): Data load failed" >> $LOG_FILE
fi

```

1.11.3 Step 3: Verify the Data Load

Run the following SQL query in PostgreSQL to check if data is loaded successfully:

```
SELECT * FROM COVID LIMIT 10;
```

If records appear, the load was successful.

```
[72]: # Showing pre-existing databases. Have quite a number of them from work but
      ↪ here are the ones within the `co` section showing no preexisting covid table
```

```
display(Image(filename="screenshots/databases_showing_no_covid.png"))
display(Image(filename="screenshots/covid_db_does_not_exist.png"))
```

circular_rocket_6897_airflow	postgres	UTF8	C	UTF-8
circular_singularity_5074_airflow	postgres	UTF8	C	UTF-8
cislunar_corona_4217_airflow	postgres	UTF8	C	UTF-8
cislunar_equinox_7302_airflow	postgres	UTF8	C	UTF-8
combusting_radiation_4547_airflow	postgres	UTF8	C	UTF-8
cometary_coordinates_9650_airflow	postgres	UTF8	C	UTF-8
cometary_element_2625_airflow	postgres	UTF8	C	UTF-8
cometary_scintillation_7266_airflow	postgres	UTF8	C	UTF-8
cometary_sunlight_1700_airflow	postgres	UTF8	C	UTF-8
create_dep_airflow	postgres	UTF8	C	UTF-8
create_ref_airflow	postgres	UTF8	C	UTF-8
create_two_three_four_airflow	postgres	UTF8	C	UTF-8
dark_horizon_9086_airflow	postgres	UTF8	C	UTF-8
dark_microwave_4664_airflow	postgres	UTF8	C	UTF-8
dark_phase_2971_airflow	postgres	UTF8	C	UTF-8
dark_universe_1432_airflow	postgres	UTF8	C	UTF-8

```
test=# \c covid_db
connection to server on socket "/tmp/.s.PGSQL.5432" failed: FATAL:  database "covid_db" does not exist
Previous connection kept
test=#
```

1.12 Task 7: Running the ETL Process and Logging

1.12.1 Step 1: Execute the Bash Script

To run the entire ETL process, execute the script:

```
./ETL_script.sh
```

This command will trigger the **Extract, Transform, and Load** phases sequentially.

1.12.2 Step 2: Monitor the Execution Logs

The script logs each stage in `ETL_log.txt`. To check the logs, run:

```
cat etl_log.txt
```

1.12.3 Step 3: Verify the Data Load

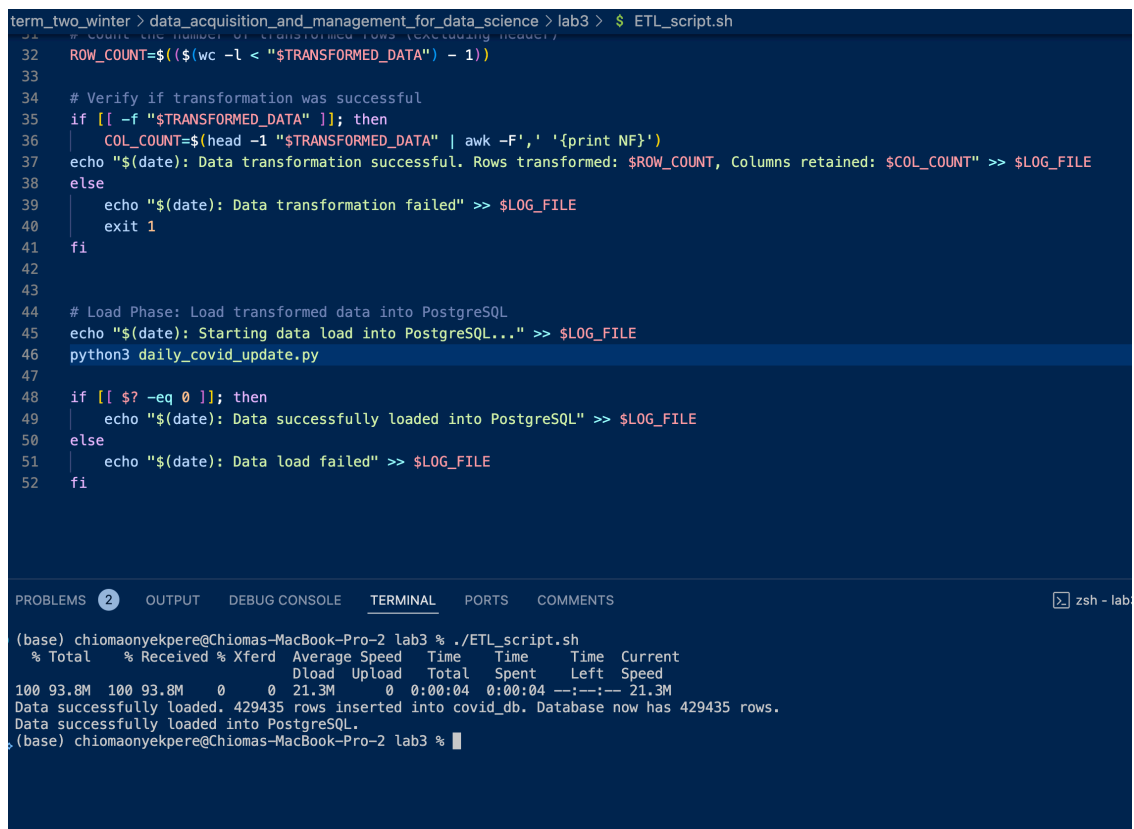
Ran the following SQL queries in PostgreSQL to check if data is loaded successfully:

```
\c covid_db;
\dt
SELECT COUNT(*) FROM COVID;
SELECT COUNT(*) FROM COVID;
```

Also confirmed that the data is being renewed and it is not adding new rows to the previous ones

[73]: *# run entire script*

```
display(Image(filename="screenshots/run_bash_script.png"))
```



```
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > $ ETL_script.sh
31 # Count the number of transformed rows (excluding header)
32 ROW_COUNT=$((wc -l < "$TRANSFORMED_DATA" - 1))
33
34 # Verify if transformation was successful
35 if [[ -f "$TRANSFORMED_DATA" ]]; then
36 | COL_COUNT=$(head -1 "$TRANSFORMED_DATA" | awk -F',' '{print NF}')
37 echo "$(date): Data transformation successful. Rows transformed: $ROW_COUNT, Columns retained: $COL_COUNT" >> $LOG_FILE
38 else
39 | echo "$(date): Data transformation failed" >> $LOG_FILE
40 | exit 1
41 fi
42
43
44 # Load Phase: Load transformed data into PostgreSQL
45 echo "$(date): Starting data load into PostgreSQL..." >> $LOG_FILE
46 python3 daily_covid_update.py
47
48 if [[ $? -eq 0 ]]; then
49 | echo "$(date): Data successfully loaded into PostgreSQL" >> $LOG_FILE
50 else
51 | echo "$(date): Data load failed" >> $LOG_FILE
52 fi

(base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 % ./ETL_script.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 93.8M  100 93.8M    0     0  21.3M      0  0:00:04  0:00:04  --:--:--  21.3M
Data successfully loaded. 429435 rows inserted into covid_db. Database now has 429435 rows.
Data successfully loaded into PostgreSQL.
(base) chiomaonyekpere@Chiomac-MacBook-Pro-2 lab3 %
```

[74]: *# confirm db data*

```
display(Image(filename="screenshots/select_covid_db.png"))
```

```

○ test=# \c covid_db
You are now connected to database "covid_db" as user "chiomaonyekpere".
covid_db=# \dt
      List of relations
  Schema | Name  | Type  | Owner
  -----+-----+-----+-----
 public | covid | table | postgres
(1 row)

covid_db=# SELECT COUNT(*) FROM COVID;
count
-----
 429435
(1 row)

covid_db=# SELECT * FROM COVID;
 iso_code | continent | date       | total_cases
-----+-----+-----+-----
 AFG      | Asia      | 2020-01-05 | 0
 AFG      | Asia      | 2020-01-06 | 0
 AFG      | Asia      | 2020-01-07 | 0
 AFG      | Asia      | 2020-01-08 | 0
 AFG      | Asia      | 2020-01-09 | 0
 AFG      | Asia      | 2020-01-10 | 0
 AFG      | Asia      | 2020-01-11 | 0
 AFG      | Asia      | 2020-01-12 | 0
 AFG      | Asia      | 2020-01-13 | 0
 AFG      | Asia      | 2020-01-14 | 0
 AFG      | Asia      | 2020-01-15 | 0
 AFG      | Asia      | 2020-01-16 | 0
 AFG      | Asia      | 2020-01-17 | 0
 AFG      | Asia      | 2020-01-18 | 0
 AFG      | Asia      | 2020-01-19 | 0
 AFG      | Asia      | 2020-01-20 | 0
 AFG      | Asia      | 2020-01-21 | 0
 AFG      | Asia      | 2020-01-22 | 0
 AFG      | Asia      | 2020-01-23 | 0
 AFG      | Asia      | 2020-01-24 | 0
 AFG      | Asia      | 2020-01-25 | 0
 AFG      | Asia      | 2020-01-26 | 0
 AFG      | Asia      | 2020-01-27 | 0
 AFG      | Asia      | 2020-01-28 | 0
 AFG      | Asia      | 2020-01-29 | 0
 AFG      | Asia      | 2020-01-30 | 0

```

1.13 Task 8: Automating the ETL Process with Cron

1.13.1 Step 1: Open the Crontab Editor

To schedule the ETL script to run daily at **10 minutes past midnight (00:10 AM)**, open the crontab editor by running:

```
crontab -e
```

1.13.2 Step 2: Add the Cron Job

Append the following line to schedule the script execution:

```
10 0 * * * /path/to/ETL_script.sh >> /path/to/etl_log.txt 2>&1
```

However, I have set mine to run every two minutes for testing purpose

```
/2 * * * * /Users/chiomaonyekpere/Documents/PACE/data-science-python-practice/term_two_winter/
```

Explanation:

- 10 0 * * * → Runs the script at **12:10 AM** every day.
- /path/to/ETL_script.sh → the **absolute path** to my ETL script.

- `>> /path/to/etl_log.txt 2>&1` → Redirects both **output** and **errors** to the log file for debugging.

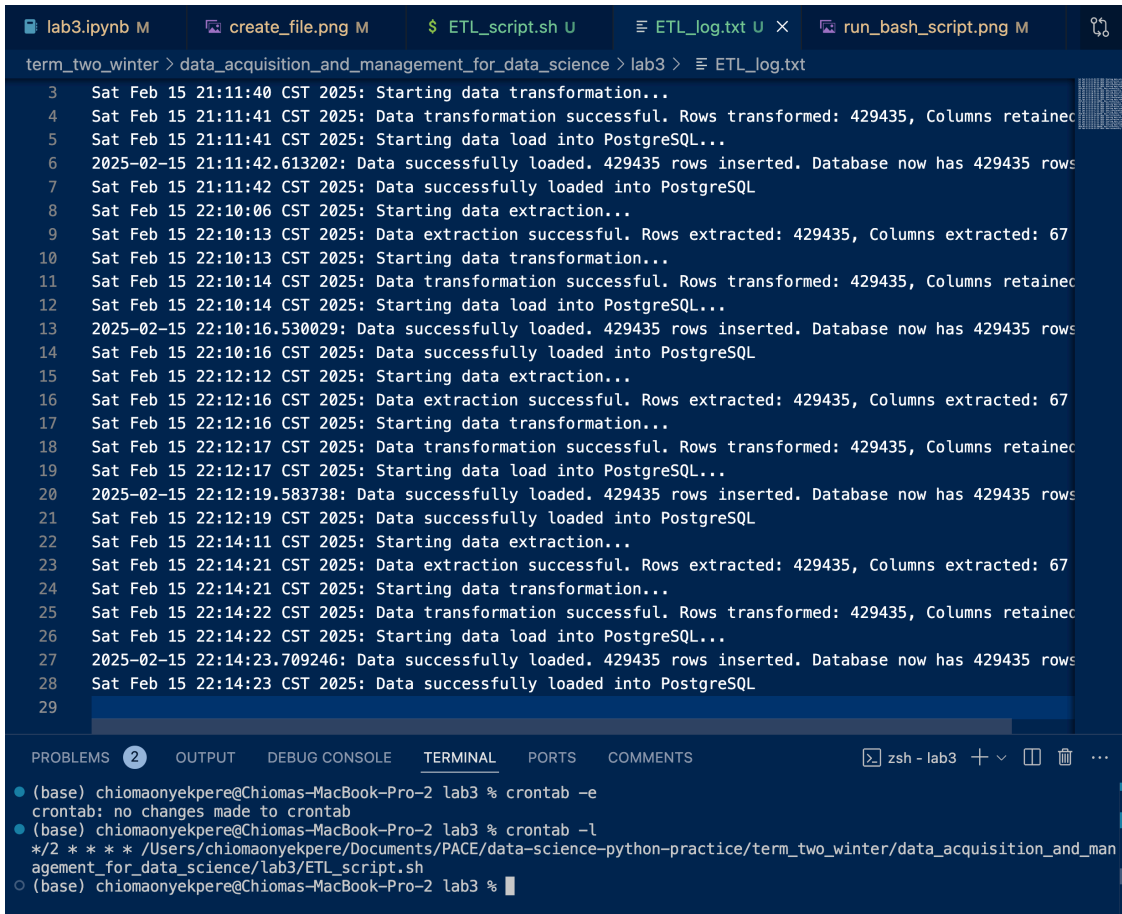
1.13.3 Step 3: Verify the Cron Job

To confirm that the cron job has been added, run:

```
crontab -l
```

This displays the scheduled job.

```
[75]: # confirm cron tab
display(Image(filename="screenshots/crontab.png"))
```



```
term_two_winter > data_acquisition_and_management_for_data_science > lab3 > ETL_log.txt
3 Sat Feb 15 21:11:40 CST 2025: Starting data transformation...
4 Sat Feb 15 21:11:41 CST 2025: Data transformation successful. Rows transformed: 429435, Columns retained
5 Sat Feb 15 21:11:41 CST 2025: Starting data load into PostgreSQL...
6 2025-02-15 21:11:42.613202: Data successfully loaded. 429435 rows inserted. Database now has 429435 rows
7 Sat Feb 15 21:11:42 CST 2025: Data successfully loaded into PostgreSQL
8 Sat Feb 15 22:10:06 CST 2025: Starting data extraction...
9 Sat Feb 15 22:10:13 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
10 Sat Feb 15 22:10:13 CST 2025: Starting data transformation...
11 Sat Feb 15 22:10:14 CST 2025: Data transformation successful. Rows transformed: 429435, Columns retained
12 Sat Feb 15 22:10:14 CST 2025: Starting data load into PostgreSQL...
13 2025-02-15 22:10:16.530029: Data successfully loaded. 429435 rows inserted. Database now has 429435 rows
14 Sat Feb 15 22:10:16 CST 2025: Data successfully loaded into PostgreSQL
15 Sat Feb 15 22:12:12 CST 2025: Starting data extraction...
16 Sat Feb 15 22:12:16 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
17 Sat Feb 15 22:12:16 CST 2025: Starting data transformation...
18 Sat Feb 15 22:12:17 CST 2025: Data transformation successful. Rows transformed: 429435, Columns retained
19 Sat Feb 15 22:12:17 CST 2025: Starting data load into PostgreSQL...
20 2025-02-15 22:12:19.583738: Data successfully loaded. 429435 rows inserted. Database now has 429435 rows
21 Sat Feb 15 22:12:19 CST 2025: Data successfully loaded into PostgreSQL
22 Sat Feb 15 22:14:11 CST 2025: Starting data extraction...
23 Sat Feb 15 22:14:21 CST 2025: Data extraction successful. Rows extracted: 429435, Columns extracted: 67
24 Sat Feb 15 22:14:21 CST 2025: Starting data transformation...
25 Sat Feb 15 22:14:22 CST 2025: Data transformation successful. Rows transformed: 429435, Columns retained
26 Sat Feb 15 22:14:22 CST 2025: Starting data load into PostgreSQL...
27 2025-02-15 22:14:23.709246: Data successfully loaded. 429435 rows inserted. Database now has 429435 rows
28 Sat Feb 15 22:14:23 CST 2025: Data successfully loaded into PostgreSQL
29

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
• (base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % crontab -e
crontab: no changes made to crontab
• (base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 % crontab -l
*/2 * * * * /Users/chiomaonyekpere/Documents/PACE/data-science-python-practice/term_two_winter/data_acquisition_and_man
agement_for_data_science/lab3/ETL_script.sh
○ (base) chiomaonyekpere@Chiomas-MacBook-Pro-2 lab3 %
```

1.14 Conclusion

This assignment successfully demonstrated the implementation of an **ETL (Extract, Transform, Load) pipeline** using **Bash scripting** and **PostgreSQL**. Throughout the process, we automated the extraction of **real-world COVID-19 data**, transformed it by selecting relevant fields, and efficiently loaded it into a structured database for analysis.

1.14.1 Key Achievements

1. Robust Data Extraction

- The pipeline ensures that the latest dataset is always downloaded, preventing outdated data from being processed.
- The number of **extracted rows and columns** is logged, improving transparency.

2. Efficient Data Transformation

- The script processes only relevant columns (`iso_code`, `continent`, `date`, `total_cases`), optimizing storage and query performance.
- The transformed dataset is validated before loading, ensuring consistency.
- The transformation step now logs both **rows transformed** and **columns retained**, improving monitoring.

3. Reliable Data Loading into PostgreSQL

- The database creation is **idempotent** (it checks if the database and table exist before creating them).
- The **database is cleared before reloading data**, preventing duplication issues.
- The script logs the **number of rows inserted** and the **total row count after insertion**, allowing for better tracking.

4. Comprehensive Logging for Debugging and Monitoring

- The ETL process logs every key step, including:
 - **Download success/failure**
 - **Number of extracted and transformed rows**
 - **Success/failure of data load**
 - **Row count before and after insertion in the database**
- This ensures transparency and easier debugging.

5. Automation with Cron Jobs

- The ETL process is fully **automated using a cron job**, ensuring that the pipeline runs **daily at 12:10 AM**.
- The job is logged for verification, and system logs are monitored for execution tracking.
- Environment variables were explicitly set to ensure the cron job runs as expected.

1.14.2 Challenges and Fixes

- **Initial Cron Job Issues** → Debugging logs were enabled, and execution paths were explicitly defined.
- **Data Overwriting Concerns** → Implemented **idempotent data loading** to clear old data before new data insertion.
- **Logging Enhancements** → Now logs **row counts, error messages, and system statuses**, providing better monitoring.

1.14.3 Future Enhancements

To further improve the ETL pipeline, we can: - Implement **data validation** before inserting records into the database. - Add **error notifications** (e.g., email alerts) when failures occur. - Optimize performance by using **batch inserts** instead of row-by-row insertion. - Extend the pipeline for **incremental updates** rather than full database reloads.

1.14.4 Final Thoughts

This ETL workflow provides a **scalable and automated solution** for handling data ingestion, transformation, and storage. By leveraging **Bash scripting, Python, and PostgreSQL**, we have created a **structured, repeatable, and efficient data pipeline**. The improved logging ensures complete visibility into each step, making it easy to track execution history and troubleshoot issues.

This assignment lays a strong foundation for **advanced data engineering projects**, where automation, logging, and database management are critical.

[]:

References

Ritchie, H., Mathieu, E., Rodés-Guirao, L., Appel, C., Giattino, C., Ortiz-Ospina, E., Hasell, J., Macdonald, B., Beltekian, D., & Roser, M. (2025). *Coronavirus Pandemic (COVID-19)*. Our World in Data[CSV file]. Retrieved from <https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv>